Project Proposal

CUDA-Based N-Body Simulation

Students: mohammad babakhani-bita bagheri

1. Objective

This project implements a **from-scratch N-body simulation using CUDA**, focusing on **performance and scalability** for computing **O(N²)** pairwise interactions. The goal is to demonstrate how GPU-oriented design decisions—thread mapping, memory layout, and shared-memory usage—enable efficient large-scale numerical simulations.

A simplified **solar system with a comet** is used to validate correctness through physically reasonable motion, while a larger particle system is used to evaluate scalability as the number of bodies increases.

2. Simulation Scale

Two configurations are used:

- **Correctness Mode:**

  12–64 bodies (Sun, planets, and a comet) to demonstrate stable orbits and gravitational deflection.

- **Benchmark Mode:**

$$N \in 1k, 2k, 4k, 8k, 16k (optionally 32k),$$

consisting of a dominant Sun, a few massive bodies, and many low-mass planetoids, enabling stress-testing of the O(N²) interaction kernel.

3. Interaction Model

Each body *i* has position $\mathbf{p_i}$, velocity $\mathbf{v_i}$, and mass $\mathbf{m_i}$. Interactions follow Newtonian gravity with softening:

$$\mathbf{a}i = \sum j \neq i G\, m_j \frac{\mathbf{p}_j - \mathbf{p}_i}{\left(\|\mathbf{p}_j - \mathbf{p}_i\|^2 + \epsilon^2\right)^{3/2}}$$

The softening term **ε** ensures numerical stability. Units may be normalized, and masses are chosen so the Sun's gravitational influence dominates. Bodies are modeled as point masses; collisions are not considered.

4. Time Integration

The simulation uses a **Leapfrog (Velocity-Verlet)** integrator with fixed timestep **Δt**, providing better orbital stability than Euler integration while remaining computationally efficient.

5. CUDA Parallelization Strategy

The entire simulation loop runs on the GPU. CPU code is used only for initial condition generation and optional visualization/output.

- **Thread Mapping:** one CUDA thread per body.
- **Data Layout:** Structure-of-Arrays or packed `float4` for coalesced memory access.
- **Kernels:**
  - $O(N^2)$ kernel for pairwise interaction/acceleration computation.
  - $O(N)$ kernels for velocity and position updates.
- **Optimization:** shared-memory tiling in the interaction kernel to reduce global memory traffic.

No physics engines or GPU-accelerated N-body libraries are used.

6. Performance Bottlenecks

The dominant cost is the $O(N^2)$ interaction kernel. Additional constraints include global memory bandwidth and host-device data transfer for visualization. These are mitigated via shared-memory tiling and infrequent data transfers.

7. Evaluation

Performance is evaluated using:

- **Time per timestep** (CUDA events)

$$Interactions per second : \text{IPS} \approx \frac{N(N-1)}{t_{\text{step}}}$$

Scaling behavior is analyzed for increasing N. Correctness is verified visually through stable planetary orbits and comet deflection.